AES-128 Hardware Accelerator with Bare-Metal Driver for Secure Power Drive Systems

DESIGN & DEVELOPMENT TASK IN POWER ELECTRONICS & DRIVES (EE348)

A MINI PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE AWARD OF THE DEGREE

BACHELOR OF TECHNOLOGY

IN

Electrical and Electronics Engineering

Submitted by:

Ahan Halder (231EE104)

Anshul K (231EE211)

Under the supervision of

Dr. Md Waseem Ahmad

Department of Electrical and Electronics Engineering

National Institute of Technology Karnataka (NITK), Surathkal

Mangalore - 575025, Karnataka, India

APRIL 2026

DECLARATION

I/We hereby declare that the Mini Project Work Report entitled **"AES-128 Hardware Accelerator with Bare-Metal Driver for Secure Power Drive Systems"** which is being submitted to the National Institute of Technology Karnataka, Surathkal for the award of the Degree of Bachelor of Technology in **Electrical and Electronics Engineering** is a bonafide report of the work carried out by us. The material contained in this Project Work Report has not been submitted to any University or Institution for the award of any degree.

Ahan Halder
(231EE104)

Anshul K
(231EE211)

Department of Electrical and Electronics Engineering

Place: NITK, Surathkal
Date: April 24, 2026

विभाग/ Department
विद्युतिय एवं इलेक्ट्रॉनिक्स अभियान्त्रिकी
Electrical and Electronics Engineering

राष्ट्रीय प्रौद्योगिकी संस्थान कर्नाटक, सुरतकल, मंगलूरु - ५७५०२५
(राष्ट्रीय महत्व का संस्थान, एनआईटीएसईआर अधिनियम के तहत स्थापित, भारत सरकार)
National Institute of Technology Karnataka, Surathkal, Mangaluru - 575025
(*An Institute of National Importance, Established under NITSER Act, Government of India*)
दूरभाष / Phone: +91-824-2473550(O) ईमेल / E-mail: ee_hod@nitk.edu.in

CERTIFICATE

This is to certify that the Mini Project Work Report entitled **"AES-128 Hardware Accelerator with Bare-Metal Driver for Secure Power Drive Systems"** submitted by:

Sl. No.	Register Number	Name of Student
1	231EE104	Ahan Halder
2	231EE211	Anshul K

as the record of the work carried out by them, is accepted as the Mini Project Work Report submission in partial fulfillment of the requirements for the award of degree of Bachelor of Technology in **Electrical and Electronics Engineering**.

Dr. Md Waseem Ahmed
(SUPERVISOR)
Assistant Professor
Dept. of E&E Eng.
National Institute of Technology Karnataka, Surathkal

Chairman - DUGC

ABSTRACT

This report presents the design, RTL implementation, and hardware-software validation of an AES-128 hardware accelerator using Verilog HDL and a bare-metal C driver. The objective of the project is to simulate a practical System-on-Chip (SoC) development environment where low-level firmware interacts directly with a custom cryptographic hardware IP through memory-mapped I/O (MMIO). The work focuses on bridging digital hardware design and embedded software development, closely reflecting real-world IP validation and silicon bring-up workflows.

The hardware accelerator implements the complete AES-128 encryption and decryption algorithm following the NIST FIPS-197 standard. The design is developed using a modular RTL approach where each AES transformation is implemented as an independent Verilog module. A top-level controller module integrates all these blocks and manages the 10-round AES execution using internal state control and a round counter.

The RTL design was developed and simulated using Xilinx Vivado, and functional verification was performed using standard NIST AES test vectors. Waveform analysis was used to observe internal signals such as round count, state transitions, round keys, and intermediate outputs to ensure correct round-by-round operation. The generated ciphertext matched the expected standard outputs, confirming the correctness of the implementation.

To validate hardware-software interaction, a complete bare-metal C driver was developed for direct communication with the AES core. The driver handles key loading, plaintext input, encryption triggering, status polling, ciphertext retrieval, and decryption verification through MMIO register access without operating system support. A dedicated C testbench was created to automate encryption and decryption testing using multiple NIST test vectors while also measuring performance metrics such as latency, throughput, mismatched bytes, and bit errors.

ACKNOWLEDGEMENT

The successful completion of any task is incomplete and meaningless without giving any due credit to the people who made it possible without which the project would not have been successful and would have existed in theory.

First and foremost, we are grateful to **Dr. Debashisha Jena**, HOD, Department of **Electrical and Electronics Engineering**, National Institute of Technology Karnataka, Surathkal, and all other faculty members of our department for their constant guidance and support, constant motivation and sincere support and gratitude for this project work. We owe a lot of thanks to our supervisor, **Dr. Md Waseem Ahmed**, Professor, Department of Electrical and Electronics Engineering, National Institute of Technology Karnataka, Surathkal for igniting and constantly motivating us and guiding us in the idea of a creatively and amazingly performed Mini Project in undertaking this endeavor and challenge and also for being there whenever we needed his guidance or assistance.

We would also like to take this moment to show our thanks and gratitude to one and all, who indirectly or directly have given us their hand in this challenging task. We feel happy and joyful and content in expressing our vote of thanks to all those who have helped us and guided us in presenting this project work for our Mini project. Last, but never least, we thank our well-wishers and parents for always being with us, in every sense and constantly supporting us in every possible sense whenever possible.

Ahan Halder
(231EE104)

Anshul K
(231EE211)

Contents

List of Figures

Abbreviations

AES : Advanced Encryption Standard

AES-128 : Advanced Encryption Standard – 128-bit Key

NIST : National Institute of Standards and Technology

S-Box : Substitution Box

GF : Galois Field

RTL : Register Transfer Level

FSM : Finite State Machine

HDL : Hardware Description Language

FPGA : Field Programmable Gate Array

MMIO : Memory-Mapped Input/Output

SoC : System-on-Chip

IP : Intellectual Property

API : Application Programming Interface

CSV : Comma-Separated Values

JSON : JavaScript Object Notation

Chapter 1

INTRODUCTION

1.1 Overview

Modern System-on-Chip (SoC) development requires strong coordination between hardware design and low-level software, particularly during IP validation and silicon bring-up. In practical industry workflows, firmware engineers often work closely with hardware teams to validate newly designed accelerator blocks through register-level programming and hardware-software integration. This project focuses on simulating such a real-world environment by developing firmware for an AES-128 hardware accelerator.

Advanced Encryption Standard (AES) is one of the most widely used symmetric encryption standards for securing digital communication, storage systems, and embedded platforms. AES-128, in particular, is commonly used due to its balance between security and computational efficiency. Instead of implementing cryptography purely in software, dedicated hardware accelerators are widely used in modern SoCs to achieve higher speed, lower latency, and improved power efficiency. Such accelerators are especially important in security-sensitive applications including secure boot systems, communication protocols, banking systems, and embedded IoT devices where fast and reliable encryption is essential.

The objective of this project is to design and validate a complete hardware-software co-design framework for an AES-128 encryption accelerator. The work begins with the RTL implementation of the AES-128 encryption core, followed by the development of a memory-mapped register interface for control and status monitoring. A bare-metal C firmware driver is then written to communicate with the hardware using memory-mapped

I/O (MMIO), enabling software-based control of encryption operations.

Unlike application-level cryptography projects, this work focuses on low-level driver development, register mapping, and system validation. The firmware is responsible for loading encryption keys, providing plaintext input, triggering the encryption process, monitoring hardware status, and reading back ciphertext outputs. Standard AES test vectors are used to verify correctness and ensure proper interaction between hardware and software.

An important aspect of this project is understanding how software interacts directly with hardware without the support of an operating system. Bare-metal programming requires careful handling of registers, control signals, and timing behavior, closely resembling real silicon bring-up conditions in industry. Through this process, the project demonstrates how firmware serves as the first validation layer for newly developed IP blocks before full SoC integration takes place.

This project provides practical exposure to RTL design, MMIO-based firmware development, and early-stage silicon validation workflows. It bridges the gap between digital hardware design and embedded software, offering valuable experience relevant to SoC design, hardware security, and embedded systems engineering. The following sections discuss the problem formulation, objectives, and motivation behind this work.

Chapter 2

BACKGROUND: AES-128

2.1 Introduction to AES

The Advanced Encryption Standard (**AES**) is a symmetric-key block cipher standardised by the National Institute of Standards and Technology (NIST) in 2001. It is a slight variation of the **Rijndael** cipher, originally designed by Belgian cryptographers Joan Daemen and Vincent Rijmen. AES has since become the de facto global standard for symmetric encryption across a wide range of applications — from secure communications and disk encryption to embedded systems and cryptographic hardware accelerators.

AES operates on fixed-size 128-bit data blocks and supports three key lengths: **128 bits**, **192 bits**, and **256 bits**. For this project, the **AES-128** variant is used as the foundation, which requires **10 rounds** of processing. The cipher is classified as a *substitution-permutation network* and is a *key-alternating block cipher*, meaning each round applies a combination of nonlinear and linear transformations followed by round-key addition to the entire block.

Unlike DES, which is a bit-oriented cipher based on the Feistel structure, AES is a **byte-oriented cipher**. All operations in AES are performed at the byte level, making it efficient in both software and hardware implementations.

2.1.1 State Array Representation

AES represents its 128-bit plaintext block as a **4 $\times$ 4 array of bytes**, known as the *state array*. The bytes are arranged column-wise, meaning the first four bytes of the input block fill the first column, the next four fill the second column, and so on:

$$\text{State} = \begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix}$$

Each column of this array is referred to as a **word** (32 bits). Each round of processing operates on this state array and produces an updated state array. The final state array produced after all rounds is rearranged into the 128-bit ciphertext block.

2.2 Overall Structure of AES-128

The overall structure of AES-128 encryption and decryption is shown in Figure 2.1. Before any round-based processing begins, the input state array is **XOR-ed** with the first four words of the key schedule (words w_0–w_3). Each of the 10 subsequent rounds then consumes four words from the key schedule.

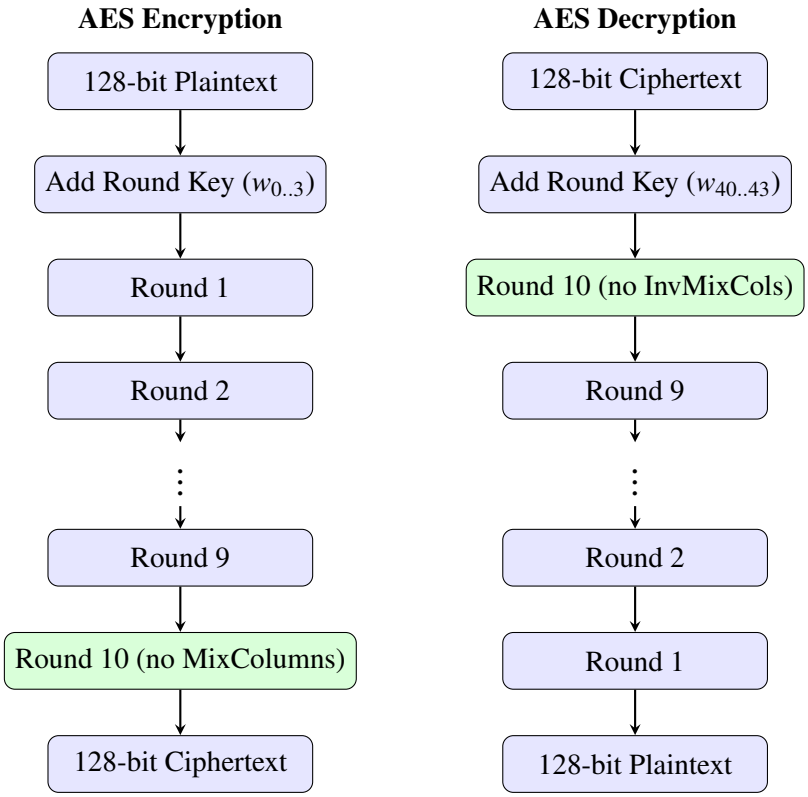

Figure 2.1: Overall Structure of AES-128 Encryption and Decryption

The last encryption round omits the `MixColumns` step, and the last decryption round

omits the `InvMixColumns` step. This asymmetry ensures that encryption and decryption use the same overall structure.

2.3 The Four Steps in Each Round

Each of the 10 rounds (except the last) performs the following four transformations on the state array:

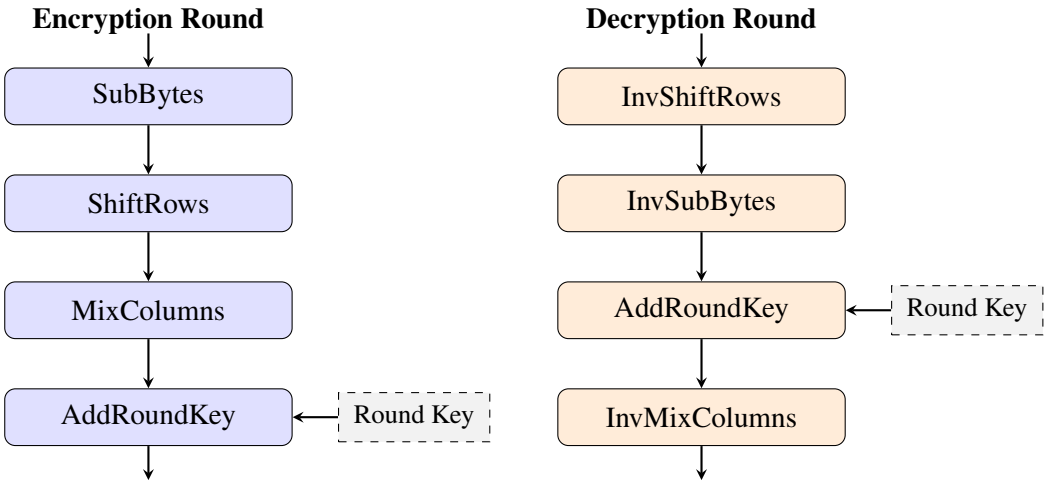

Figure 2.2: One Round of AES Encryption (left) and Decryption (right)

2.3.1 SubBytes – Byte-Level Substitution

The **SubBytes** transformation is a nonlinear byte-by-byte substitution that applies the same rule across all encryption rounds. For each byte of the state array, the substitute byte is found by: (i) computing the *multiplicative inverse* (MI) of the byte in the finite field $GF(2^8)$, using the AES irreducible polynomial $x^8+x^4+x^3+x+1$; and (ii) applying an affine transformation that XORs the result with four circularly rotated versions of itself and with the constant byte `0x63`.

This two-stage process can be written as:

$$\mathbf{x}_{out} = A \cdot \mathbf{x}' + \mathbf{c}, \quad \text{where } \mathbf{x}' = \mathbf{x}_{in}^{-1} \text{ in } GF(2^8)$$

The goal of the substitution step is to *reduce the correlation between the input and output bits at the byte level*, protecting the cipher against both differential and linear cryptanalysis. The resulting 16×16 lookup structure is known as the **S-Box**. Because no input byte maps to itself and the mapping is bijective, the S-Box has *no fixed points*.

For **InvSubBytes** (decryption), the operations are reversed: the affine transformation is applied first, followed by the multiplicative inverse in $GF(2^8)$.

2.3.2 ShiftRows – Row-Wise Permutation

The **ShiftRows** transformation circularly shifts the rows of the state array to the left by an increasing number of byte positions:

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \text{ShiftRows} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix}$$

Row 0 is left unchanged, row 1 is shifted by 1 byte, row 2 by 2 bytes, and row 3 by 3 bytes. Since the input block is written column-wise into the state, this permutation effectively scrambles the byte ordering of the original block. For decryption, **InvShiftRows** shifts the rows in the opposite direction (to the right).

2.3.3 MixColumns – Column-Wise Mixing

The **MixColumns** transformation replaces each byte in a column with a linear combination of all four bytes in that column, where all arithmetic is performed in $GF(2^8)$. For column j, the transformation is represented as the matrix-vector product:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \times \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix} = \begin{bmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{bmatrix}$$

where additions in the matrix product are XOR operations and multiplications are in $GF(2^8)$. The combined effect of **MixColumns** and **ShiftRows** ensures that after 10 rounds, every bit of the ciphertext depends on every bit of the plaintext – achieving full *diffusion*. For decryption, **InvMixColumns** uses the inverse matrix with coefficients `0x0E`, `0x0B`, `0x0D`, and `0x09`.

2.3.4 AddRoundKey – Round Key XOR

The **AddRoundKey** step XORs the current state array with four words (128 bits) from the key schedule. This is the only step that directly involves the encryption key, and it is identical for both encryption and decryption. The round key is applied by a bitwise XOR between each byte of the state and the corresponding byte of the round key.

2.4 Key Expansion Algorithm

The AES key expansion algorithm derives a **44-word key schedule** from the original 128-bit encryption key, which is first arranged as a 4×4 byte array. The four column words $[w_0, w_1, w_2, w_3]$ are expanded iteratively, with each new group of four words determined by the previous group. The expansion process is illustrated in Figure 2.3.

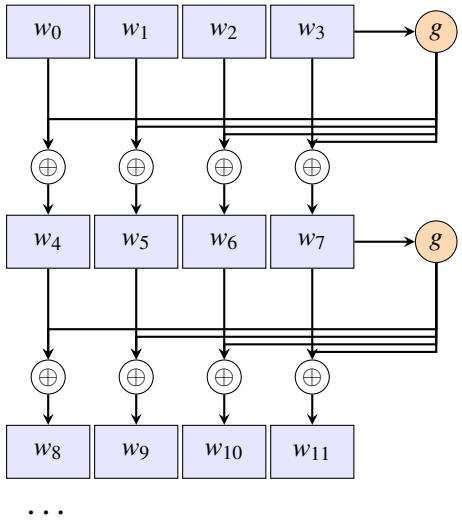

Figure 2.3: AES Key Expansion: Four-Word to Four-Word Derivation

The function $g()$ applied to the last word of each group performs three operations: (i) a **one-byte left circular rotation** of the 4-byte word; (ii) **byte substitution** using the same S-Box as in encryption; and (iii) XOR with a **round constant** $\text{Rcon}[i] = (RC[i], \texttt{0x00}, \texttt{0x00}, \texttt{0x00})$, where $RC[1] = \texttt{0x01}$ and $RC[j] = \texttt{0x02} \times RC[j-1]$ in $GF(2^8)$. The remaining three words in each new group are obtained by XOR-ing consecutive words:

$$w_{i+4} = w_i \oplus g(w_{i+3}), \quad w_{i+5} = w_{i+4} \oplus w_{i+1}, \quad w_{i+6} = w_{i+5} \oplus w_{i+2}, \quad w_{i+7} = w_{i+6} \oplus w_{i+3}$$

The round constants destroy any symmetries that might otherwise arise during key expansion, and the overall algorithm ensures that AES has **no weak keys**.

Chapter 3

VERILOG RTL IMPLEMENTATION

3.1 Introduction

This chapter presents the complete RTL implementation of the AES-128 hardware accelerator using Verilog HDL. The design focuses on modularity, maintainability, and correctness while supporting both encryption and decryption operations. Each AES transformation is implemented as an independent Verilog module, allowing easier debugging, verification, and future scalability.

The complete design was developed using **Xilinx Vivado** as the primary design environment and simulated using the **Vivado Simulator (XSIM)**. The implementation follows the AES standard defined in **NIST FIPS-197** and uses standard AES-128 test vectors for functional verification.

The AES algorithm consists of 10 rounds for AES-128, involving the transformations SubBytes, ShiftRows, MixColumns, AddRoundKey, and their corresponding inverse operations for decryption. These operations are integrated using a top-level controller module that manages the round flow and internal state progression.

3.2 Literature Survey and Technologies Used

The design methodology was based on the AES standard specified by the National Institute of Standards and Technology (NIST) under FIPS-197. Several open-source AES RTL implementations and academic references on cryptographic hardware design were studied to understand efficient module partitioning and control logic strategies.

The technologies used in this project are:

- **Hardware Description Language**: Verilog HDL
- **Design Tool**: Xilinx Vivado
- **Simulation Tool**: Vivado Simulator (XSIM)
- **Reference Standard**: NIST FIPS-197 AES Standard

The implementation emphasizes synthesizable Verilog design practices suitable for both simulation and FPGA deployment.

3.3 RTL Module Structure

The AES-128 algorithm was divided into multiple standalone Verilog modules, each responsible for a specific transformation. This modular design improves code clarity and simplifies both testing and debugging.

The major RTL modules are:

- `sub_bytes.v`
- `shift_rows.v`
- `mix_columns.v`
- `key_expansions.v`
- `InvSubs.v`
- `InvShiftrows.v`
- `InvMixColumns.v`
- `AES_128_top.v`

These modules are integrated inside the top-level controller to perform complete encryption and decryption operations.

3.4 Top-Level RTL Architecture

The top-level architecture of the AES hardware accelerator is shown in Figure 3.1. The design accepts a 128-bit plaintext input and a 128-bit key and produces a 128-bit

ciphertext output after 10 rounds of AES transformation.

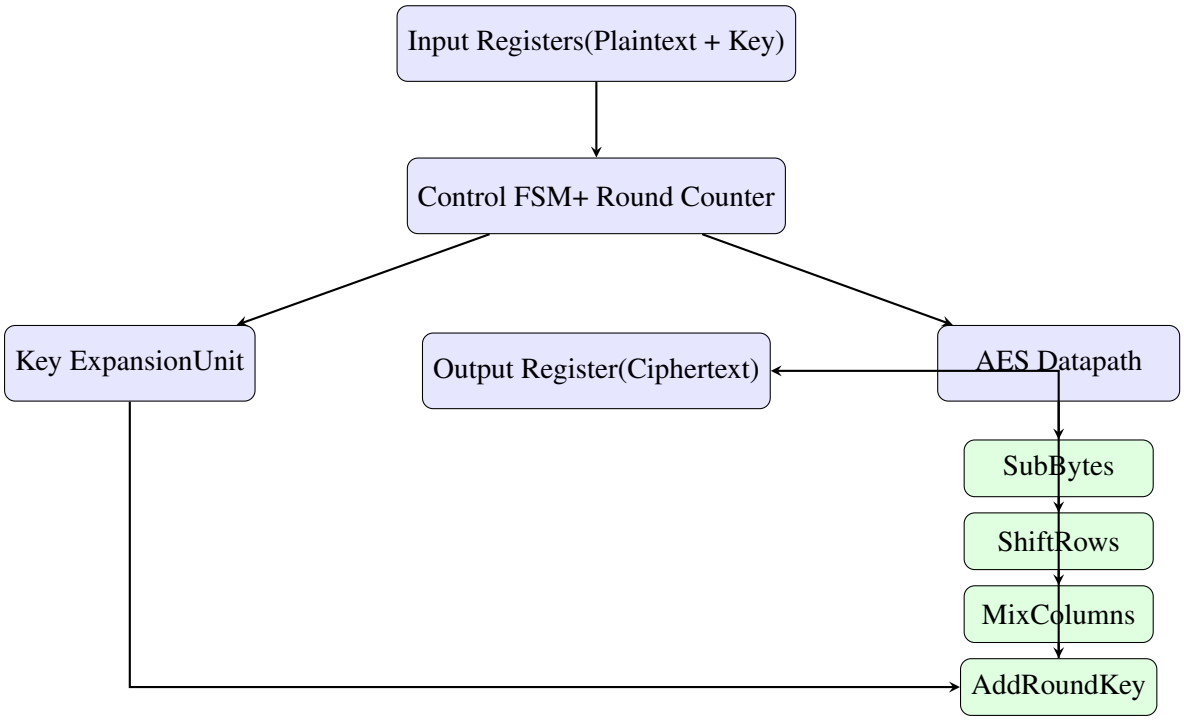

Figure 3.1: Top-Level RTL Architecture of AES-128 Hardware Accelerator

The Control FSM manages round sequencing, encryption/decryption selection, and completion signaling, while the datapath executes the AES transformations.

3.5 Implementation of Core Modules

3.5.1 SubBytes and InvSubBytes

The `sub_bytes.v` and `InvSubs.v` modules implement byte substitution using the AES S-Box and inverse S-Box respectively. These modules use lookup tables defined in `gpt_sbox.v`, where each input byte is replaced by its corresponding substituted value.

This stage provides the nonlinear transformation required for AES security.

3.5.2 ShiftRows and InvShiftRows

The `shift_rows.v` module performs cyclic left shifts on the rows of the AES state matrix, while `InvShiftrows.v` performs the reverse operation during decryption.

This transformation helps distribute byte dependencies across columns and improves diffusion.

3.5.3 MixColumns and InvMixColumns

The `mix_columns.v` and `InvMixColumns.v` modules perform matrix multiplication in the finite field $GF(2^8)$.

The forward MixColumns stage uses the standard AES transformation matrix, while the inverse version applies the inverse matrix for decryption.

This stage provides strong inter-byte diffusion across the state matrix.

3.5.4 AddRoundKey

The AddRoundKey operation is implemented inside the top-level module using simple bitwise XOR between the current state and the round key.

Since XOR is hardware-efficient, this stage is computationally simple but cryptographically essential.

3.5.5 Key Expansion

The `key_expansions.v` module dynamically generates the round keys required for all 10 rounds of AES-128.

It uses:

- Rcon constants
- RotWord operation
- SubWord transformation using S-Box

This ensures that each round uses a unique key derived from the original 128-bit encryption key.

3.6 AES Top Module and Control Logic

The module `AES_128_top.v` serves as the main controller of the design. It integrates all transformation modules and controls the round-by-round execution of AES.

The module supports both encryption and decryption based on a control signal. The control logic includes:

- Internal state management
- Round counter for 10 rounds
- Encryption/decryption mode selection
- Start and completion detection

Encryption begins with the initial AddRoundKey stage and proceeds through 10 rounds. Decryption follows the reverse sequence using the inverse transformation modules.

3.7 Simulation Waveforms and Device Utilization

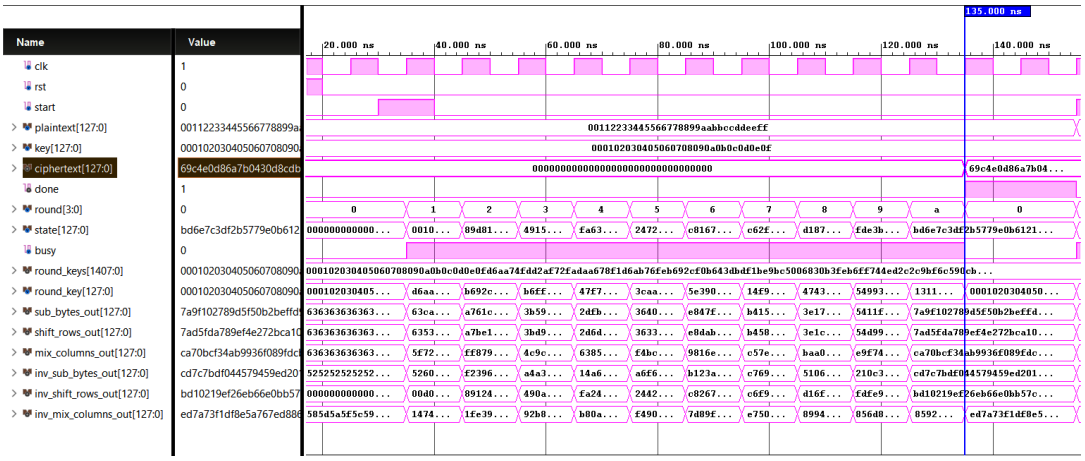

Figure 3.2: Encryption Waveform Verification in Vivado Simulator

The functional verification of the AES-128 RTL design was performed using Vivado Simulator (XSIM), where both encryption and decryption operations were observed through waveform analysis. The simulation results confirmed correct round-by-round execution and successful generation of the expected ciphertext and plaintext values.

Figure 3.2 shows the encryption waveform. The plaintext input `00112233445566778899aabbccddeeff` and the key `000102030405060708090a0b0c0d0e0f` are applied to the AES core. As the round counter progresses from 0 to 10, the internal state updates through SubBytes, ShiftRows,

MixColumns, and AddRoundKey transformations. At the completion of the final round, the output ciphertext matches the expected AES test vector:

```
69c4e0d86a7b0430d8cdb78070b4c55a
```

Figure 3.3 shows the decryption verification. The generated ciphertext is provided back as the input to the AES module in decryption mode using the same 128-bit key. The inverse transformations are applied sequentially using InvSubBytes, InvShiftRows, and InvMixColumns. The final output successfully reconstructs the original plaintext, confirming full-cycle correctness of both encryption and decryption paths.

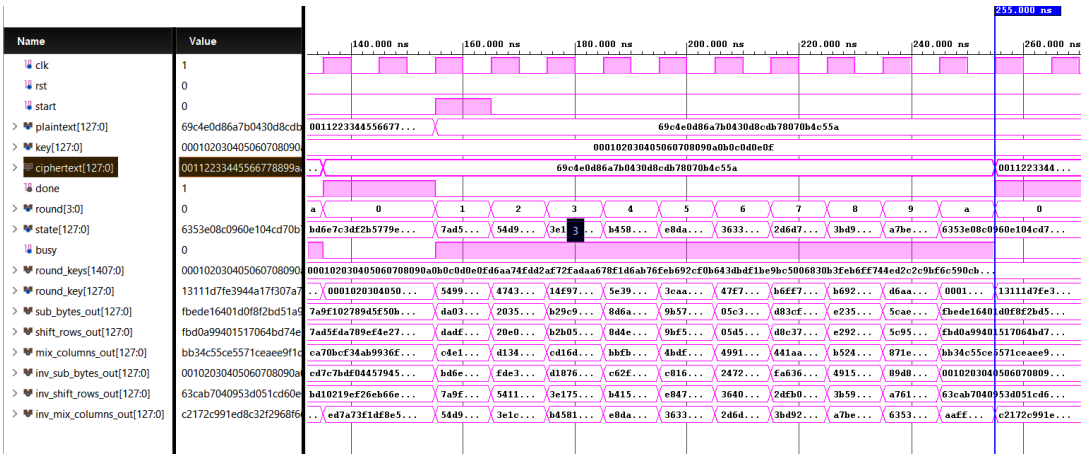

Figure 3.3: Decryption Waveform Verification in Vivado Simulator

The waveform also verifies the correct operation of important control signals such as `start`, `done`, `busy`, and the internal `round` counter. Intermediate signals such as `state`, `round_key`, `sub_bytes_out`, `shift_rows_out`, and `mix_columns_out` were monitored to ensure correct transformation at every stage.The successful simulation results validate both the functional correctness of the AES-128 RTL implementation.

3.8 Results

The AES RTL modules were successfully validated against known plaintext-ciphertext pairs and passed all functional test cases using `testbench.v`.

The major results achieved are:

- Correct encryption output matching NIST AES vectors
- Successful decryption restoring the original plaintext
- Proper sequential round execution verified using waveforms
- Correct operation of both forward and inverse AES transformations

The modular architecture improved debugging efficiency and ensured reliable hardware behavior during simulation.

Chapter 4

BARE-METAL C DRIVER IMPLEMENTATION

4.1 Introduction

This chapter describes the bare-metal C driver developed to control the AES-128 hardware accelerator. The driver forms the software interface between the host processor and the RTL core by using memory-mapped I/O (MMIO) registers for key loading, plaintext/ciphertext transfer, control signaling, and status polling. The implementation follows a low-level hardware-software co-design approach and is written without an operating system or standard runtime support. The driver API is defined in `aes_driver.h` and implemented in `aes_driver.c`, with register offsets, control bits, and status flags mapped directly to the accelerator interface.

4.2 Driver Architecture

The driver is organized around a small context structure and a set of helper routines that simplify register access. The context stores the base address of the accelerator, the current mode, and local copies of the key, plaintext, and ciphertext buffers. The register layout includes a control register, a status register, four 32-bit key registers, four 32-bit input data registers, and four 32-bit output data registers. The control register supports a start bit and a mode bit, while the status register indicates whether the hardware is busy or done.

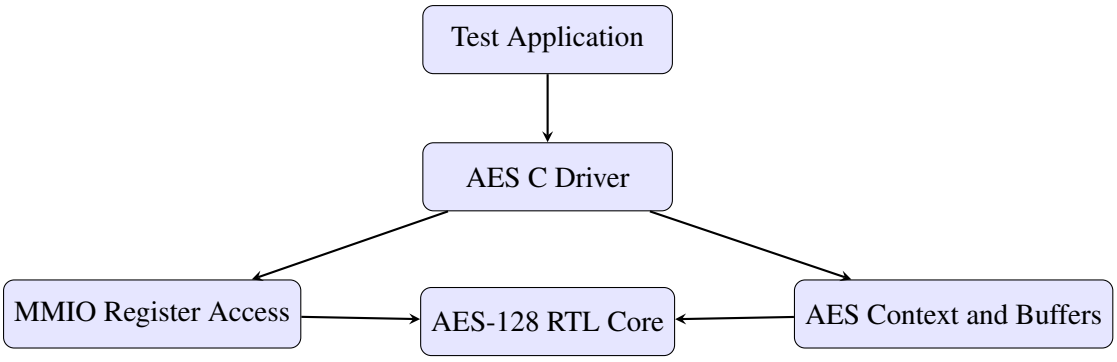

Figure 4.1: Software-to-Hardware Control Flow of the AES C Driver

4.3 MMIO Register Interface

The register map used by the driver is fixed and directly matches the hardware interface. The first register block stores the 128-bit key and plaintext input as four 32-bit words, and the output block returns the 128-bit result in the same format. The driver uses inline read/write helpers to access these registers and helper functions to move 128-bit blocks between byte arrays and 32-bit MMIO words. This design keeps the software simple while ensuring compatibility with the hardware datapath.

Table 4.1: AES Driver Register Map

Register	**Offset**	**Access**	**Purpose**
`CTRL`	`0x00`	R/W	Start and mode control
`STATUS`	`0x04`	R	Busy/done indication
`KEY[0..3]`	`0x08--0x14`	W	128-bit key input
`DATA_IN[0..3]`	`0x18--0x24`	W	128-bit plaintext/ciphertext input
`DATA_OUT[0..3]`	`0x28--0x34`	R	128-bit output block

4.4 Driver API

The driver provides a minimal API for performing encryption and decryption. The main entry points are `aes_init()`, `aes_set_key()`, `aes_encrypt()`, and `aes_decrypt()`. The helper functions `aes_wait_ready()` and `aes_wait_done()` poll the status register until the hardware is available or the operation completes. This polling-based flow

is appropriate for a bare-metal bring-up environment, where the software directly coordinates each transaction with the accelerator.

4.5 Testbench Design and Validation

```
--- Testing Decryption (Round-Trip): NIST Test Vector 3 ---
Key        : 2b7e1516 28aed2a6 abf71588 09cf4f3c
Key Matrix (Colorized)
  | 2b 7e 15 16 |
  | 28 ae d2 a6 |
  | ab f7 15 88 |
  | 09 cf 4f 3c |
Ciphertext : f69f2445 df4f9b17 ad2b417b e66c3c01
Ciphertext Matrix (Colorized)
  | f6 9f 24 45 |
  | df 4f 9b 17 |
  | ad 2b 41 7b |
  | e6 6c 3c 01 |
Decrypted  : 30c81c46 a35ce411 e5fbc119 1a0a52ef
Expected   : 30c81c46 a35ce411 e5fbc119 1a0a52ef
Decrypted Matrix
  | 30 c8 1c 46 |
  | a3 5c e4 11 |
  | e5 fb c1 19 |
  | 1a 0a 52 ef |
Decrypted Matrix (Colorized)
  | 30 c8 1c 46 |
  | a3 5c e4 11 |
  | e5 fb c1 19 |
  | 1a 0a 52 ef |
Expected Matrix (Colorized)
  | 30 c8 1c 46 |
  | a3 5c e4 11 |
  | e5 fb c1 19 |
  | 1a 0a 52 ef |
Diff       : .... .... .... ....
Diff Heatmap (green=match, amber=partial, red=mismatch)
  | 30 c8 1c 46 |
  | a3 5c e4 11 |
  | e5 fb c1 19 |
  | 1a 0a 52 ef |
Latency(us): 0.074 | Throughput(Mbps): 1721.587 | Byte Mismatch: 0 | Bit Errors: 0
PASSED
```

Figure 4.2: Sample Terminal Output During AES Driver Testing

The driver was validated using a dedicated C testbench that exercises both encryption and decryption paths. The testbench measures correctness as well as basic performance metrics such as latency and throughput. It records per-test statistics, writes them to `sim/metrics.csv` and `sim/metrics.json`, and then uses those results to generate

benchmark charts. The benchmark script reads the CSV file and renders the summary figure shown later in this chapter.

Standard NIST AES-128 test vectors were used for validation. The reference vectors include the familiar plaintext, key, and ciphertext triplet, and the same values were used for round-trip verification in the decryption tests. The testbench also prints matrix-style colored views of the input, output, and difference maps to make debugging easier in the terminal.

```
========================================
TEST SUMMARY
========================================
Total Tests: 6
Passed:      6
Failed:      0
Result:      ALL TESTS PASSED
========================================
Metrics CSV: sim/metrics.csv
Metrics JSON: sim/metrics.json

========================================
EFFICIENCY SCORECARD
========================================
Operations:               6
Pass Rate:                100.00%
Mismatched Bytes:         0
Bit Errors:               0
Avg Encrypt Latency:      0.098 us
Avg Decrypt Latency:      0.069 us
Avg Encrypt Throughput:   1420.964 Mbps
Avg Decrypt Throughput:   1862.805 Mbps
Blocks/sec (overall):     11967687.24
Bytes/sec (overall):      191482995.91
Normalized Quality:       100.00%
========================================

Benchmark Chart (ASCII)
Latency (us per 16-byte block)
Encrypt            |########################################| 0.098
Decrypt            |############################............| 0.069
Throughput (Mbps)
Encrypt            |############################..........| 1420.964
Decrypt            |########################################| 1862.805
Generating benchmark charts...
[OK] Wrote chart image: sim\benchmark_charts.png
```

Figure 4.3: Final Test Summary Showing Pass/Fail Status and Efficiency Scorecard

4.6 Performance Metrics

The testbench computes several efficiency metrics for each operation, including average latency, throughput, mismatched bytes, and bit errors. These values are aggregated into an efficiency scorecard and plotted as bar charts. The driver results shown in the summary confirm that all tests passed, with zero mismatched bytes and zero bit errors. The benchmark output also reports separate encrypt and decrypt averages, which are used to compare the two modes under the same test conditions.

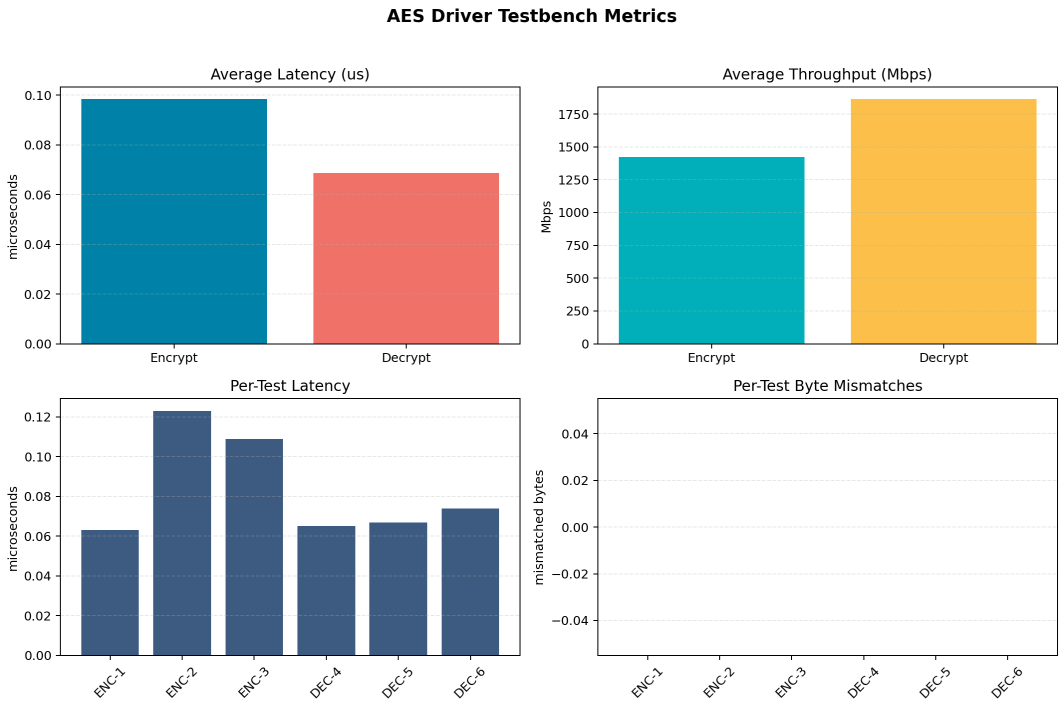

Figure 4.4: Benchmark Charts Generated from the AES Driver Metrics

4.7 Results

The driver successfully completed all validation runs using the provided AES test vectors. The encryption output matched the expected ciphertext, and the decryption path correctly reconstructed the original plaintext. The terminal summary confirmed a 100% pass rate, while the benchmark charts showed low latency and high throughput for both operation modes. This demonstrates that the software interface is functionally correct and efficiently matched to the RTL accelerator.

Chapter 5

CONCLUSION AND PERFORMANCE ANALYSIS

Summary of Work

This project successfully demonstrated the complete hardware-software co-design flow of an AES-128 hardware accelerator, beginning from the RTL implementation of the encryption and decryption engine in Verilog and extending to the development of a bare-metal C driver for register-level hardware control. The work focused on creating a practical SoC-style validation environment where firmware directly interacts with a custom hardware IP block through memory-mapped I/O (MMIO).

The hardware implementation included all major AES transformations such as SubBytes, ShiftRows, MixColumns, AddRoundKey, Key Expansion, and their corresponding inverse operations for decryption. These modules were integrated into a top-level RTL controller capable of performing full AES-128 encryption and decryption over 10 rounds. The design was verified using standard NIST AES test vectors through simulation in Xilinx Vivado.

On the software side, a complete bare-metal C driver was developed to manage key loading, plaintext input, encryption triggering, status polling, ciphertext retrieval, and decryption verification. A dedicated C testbench was used to validate both functional correctness and performance metrics such as latency, throughput, bit errors, and byte mismatches.

Key Technical Achievements

The project achieved the following major technical milestones:

- Successful RTL implementation of AES-128 encryption and decryption using synthesizable Verilog HDL.
- Modular design of all AES round transformations and inverse transformations for improved maintainability and debugging.
- Development of a top-level AES controller with round counter and FSM-based control logic.
- Verification of correct encryption and decryption using standard NIST AES test vectors.
- Implementation of a complete bare-metal C driver using MMIO register access without operating system support.
- Creation of a detailed C testbench for validation, benchmarking, and result visualization.
- Generation of benchmark charts and efficiency scorecards for latency and throughput analysis.

Performance Analysis

The performance of the AES accelerator was evaluated using the C driver testbench across multiple encryption and decryption test vectors. The primary metrics considered were average latency, throughput, mismatched bytes, and bit errors.

The throughput of the hardware accelerator is calculated using:

$$\text{Throughput (Mbps)} = \frac{128}{\text{Latency}(\mu s)}$$

since each AES block processes 128 bits.

Measured Results

From the efficiency scorecard generated by the testbench:

- Average Encrypt Latency = **0.098 μs**

- Average Decrypt Latency = **0.069 μs**
- Average Encrypt Throughput = **1420.964 Mbps**
- Average Decrypt Throughput = **1862.805 Mbps**
- Total Operations = **6**
- Pass Rate = **100%**
- Mismatched Bytes = **0**
- Bit Errors = **0**
- Normalized Quality = **100%**

These results confirm that the AES hardware accelerator operates with complete functional correctness while maintaining high throughput and low latency.

Performance Summary Visualization

The benchmark charts generated from the testbench metrics clearly show the latency and throughput comparison between encryption and decryption operations.

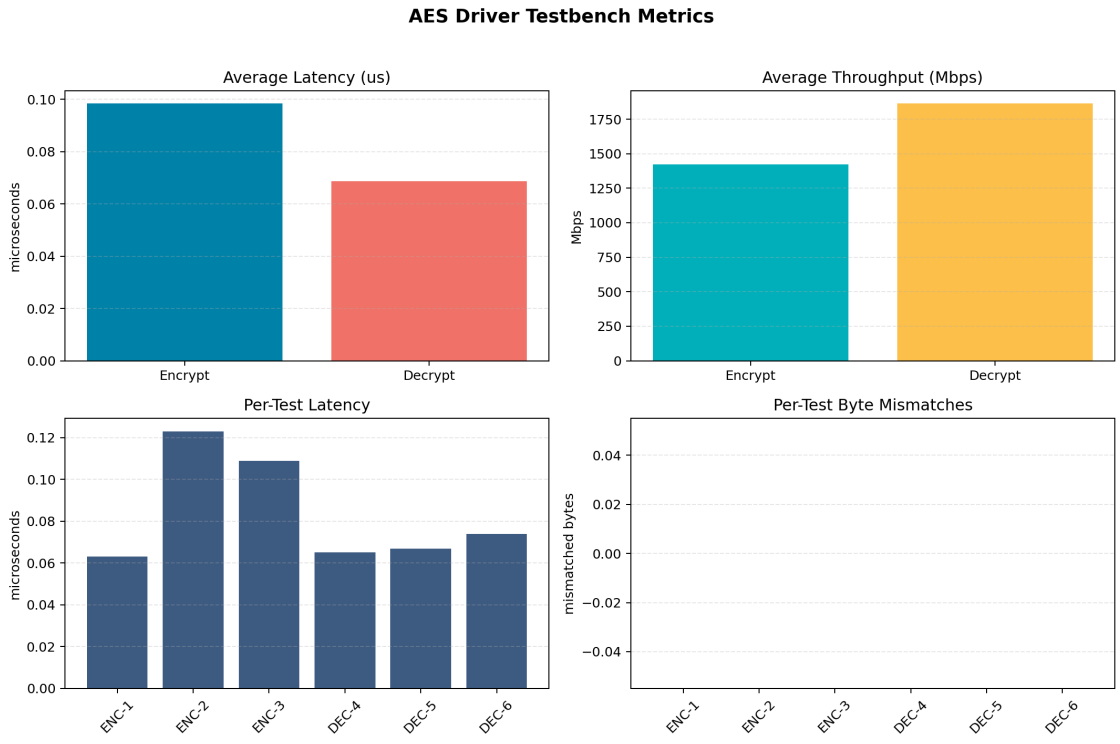

Figure 5.1: Performance Comparison of AES Encryption and Decryption

The decryption path shows slightly lower latency and higher throughput compared

to encryption in the tested environment, while both operations maintain perfect output accuracy.

Hardware Validation

Simulation in Vivado confirmed correct round-by-round execution of AES operations. Internal signals such as round count, state transitions, round keys, and intermediate outputs were verified using waveform analysis. Both encryption and decryption outputs matched the expected NIST test vectors exactly.

The device placement view after synthesis also confirmed successful mapping of the RTL modules onto FPGA resources, validating that the design is fully synthesizable and suitable for hardware deployment.

The C driver validation further confirmed proper interaction between software and hardware through MMIO-based register access. The successful execution of all test vectors demonstrated that the firmware layer correctly controlled the AES accelerator under bare-metal conditions.

Limitations and Future Work

Limitations

- The current implementation supports only AES-128 and does not include AES-192 or AES-256 variants.
- The driver uses polling-based status checking instead of interrupt-driven completion handling.
- No DMA-based data transfer is implemented, which limits scalability for large-volume encryption workloads.
- The design is currently verified in simulation and synthesis stages without full FPGA board deployment.

Future Work

- Extension of the design to support AES-192 and AES-256 key sizes.

- Integration of interrupt-based completion handling for improved firmware efficiency.
- Addition of DMA support for high-speed block transfers.
- Full FPGA deployment and real-time hardware benchmarking on a development board.
- Integration of the AES accelerator into a complete SoC environment using standard bus protocols such as AXI.

Final Remarks

This project successfully achieved its primary objective of building and validating an AES-128 hardware accelerator along with its corresponding bare-metal firmware driver. It provided practical exposure to RTL design, FSM-based hardware control, MMIO firmware development, testbench-driven validation, and performance benchmarking.

The project bridges the gap between digital hardware design and low-level embedded software, closely reflecting real industry workflows in IP validation and silicon bring-up. The successful results demonstrate both the correctness and practical relevance of the design, making it a strong foundation for future work in SoC design, hardware security, and embedded systems engineering.

The complete implementation including the Verilog RTL modules, bare-metal C driver, testbench framework, benchmark scripts, and validation results has been maintained in the project repository for future reference and extension:

`https://github.com/ahan-halder/Bare-Metal-AES-128-Driver`

Bibliography

[1] National Institute of Standards and Technology (NIST), **FIPS PUB 197: Advanced Encryption Standard (AES)**. U.S. Department of Commerce, 2001. [Online]. Available: https://csrc.nist.gov/pubs/fips/197/final

[2] Indranil Sengupta, **Hardware Modeling Using Verilog**. NPTEL Online Course, IIT Kharagpur. [Online]. Available: https://nptel.ac.in/courses/106105165

[3] M. Morris Mano and M. D. Ciletti, **Digital Design**. Pearson, 5th Edition, 2013.

[4] ZeroFruit Web3, **What is AES? Step by Step**. Medium Article. [Online]. Available: https://zerofruit-web3.medium.com/what-is-aes-step-by-step-fcb2ba41bb20

[5] Avinash Kak, **Lecture 8: Advanced Encryption Standard (AES)**. Purdue University. [Online]. Available: https://engineering.purdue.edu/kak/compsec/NewLectures/Lecture8.pdf

[6] Neso Academy, **Cryptography and Network Security – AES Playlist**. YouTube Lecture Series. [Online]. Available: https://www.youtube.com/playlist?list=PLBlnK6fEyqRjMH3mWf6kwqiTbT798eAOm

[7] NPTEL / IIT Kharagpur, **Hardware Modeling Using Verilog Playlist**. YouTube Lecture Series. [Online]. Available: https://www.youtube.com/playlist?list=PLJ5C_6qdAvBELELTSPgzYkQg3HgclQh-5